

PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 3

Introduction to OOPS Programming

MINI PROJECT 3

“Expense Tracker DB” Description: Design the backend for a Personal Expense Tracker App. Create users, transactions, and categories tables. Write SQL scripts for CRUD operations. Add a stored procedure for monthly summary generation. **Trainer Deliverables:** Students should submit .sql file with queries. Test queries via MySQL Workbench live.

TASKS :

1. Create a MySQL table called **playlists** to store Spotify-style music playlists, with columns: **playlist_id** (primary key), **user_id** (foreign key), **name** (varchar), and **created_at** (datetime). Write the **CREATE TABLE** statement.

→ CODE:

Creating users table

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY,  
    username VARCHAR(100)  
) ENGINE = InnoDB;
```

Inserting some users

```
INSERT INTO users (user_id, username)  
VALUES  
(101, 'Haaris'),  
(102, 'John'),  
(103, 'Alex');
```

Creating Playlists table

```
CREATE TABLE playlists (  
  playlist_id INT PRIMARY KEY,  
  user_id INT,  
  name VARCHAR(100),  
  created_at DATETIME,  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
) ENGINE = InnoDB;
```

Checking the table

```
DESCRIBE playlists;
```

OUTPUT:

Field	Type	Null	Key	Default	Extra
playlist_id	int(11)	NO	PRI	NULL	
user_id	int(11)	YES	MUL	NULL	
name	varchar(100)	YES		NULL	
created_at	datetime	YES		NULL	

2. Insert three sample records into the playlists table: one for a user with a 'Workout Mix', one with a 'Chill Vibes', and one with a 'Top Hits' playlist. Use realistic data for each field.

➔ CODE:

```
INSERT INTO playlists  
(playlist_id, user_id, name, created_at)  
VALUES  
(1, 101, 'Workout Mix', '2026-09-03 08:30:00'),  
(2, 102, 'Chill Vibes', '2026-09-03 19:15:00'),  
(3, 103, 'Top Hits', '2026-09-02 21:00:00');
```

OUTPUT:

				playlist_id	user_id	name	created_at
☐				1	101	Workout Mix	2026-09-03 08:30:00
☐				2	102	Chill Vibes	2026-09-03 19:15:00
☐				3	103	Top Hits	2026-09-02 21:00:00

3. Write an SQL UPDATE statement to rename the 'Chill Vibes' playlist to 'Evening Chill' for a specific user_id.

➔ CODE:

Updating playlist for user_id = 102

```
UPDATE playlists
```

```
SET name = 'Evening Chill'
```

```
WHERE user_id = 102
```

```
AND name = 'Chill Vibes';
```

Checking results

```
SELECT * FROM playlists
```

```
WHERE user_id = 102;
```

OUTPUT:

	playlist_id	user_id	name	created_at
☐  Edit  Copy  Delete	2	102	Evening Chill	2026-09-03 19:15:00

4. Write a DELETE statement to remove a playlist named 'Workout Mix' for a given user_id from the playlists table.

Hint: Make sure your WHERE clause targets only the intended row.

➔ CODE:

```
DELETE FROM playlists
```

```
WHERE user_id = 101
```

```
AND name = 'Workout Mix';
```

Checking the results

```
SELECT * FROM playlists;
```

OUTPUT:

	playlist_id	user_id	name	created_at
☐  Edit  Copy  Delete	2	102	Evening Chill	2026-09-03 19:15:00
☐  Edit  Copy  Delete	3	103	Top Hits	2026-09-02 21:00:00

5. Create a stored procedure named `GetMonthlyPlaylistCount` that takes a `user_id` and a month (as an integer) as input, and returns the total number of playlists the user created in that month.
- Hint: Use the `MONTH()` function in your `WHERE` clause.

➔ **CODE:**

```
DELIMITER //
```



```
CREATE PROCEDURE GetMonthlyPlaylistCount(  
    IN p_user_id INT,  
    IN p_month INT  
)  
BEGIN  
    SELECT COUNT(*) AS total_playlists  
    FROM playlists  
    WHERE user_id = p_user_id  
    AND MONTH(created_at) = p_month;  
END //
```



```
DELIMITER ;
```

Testing the procedure

For users 102 and September (9):

```
CALL GetMonthlyPlaylistCount(102, 9);
```

OUTPUT:

total_playlists

1
